

## Module 12 - chapter 18

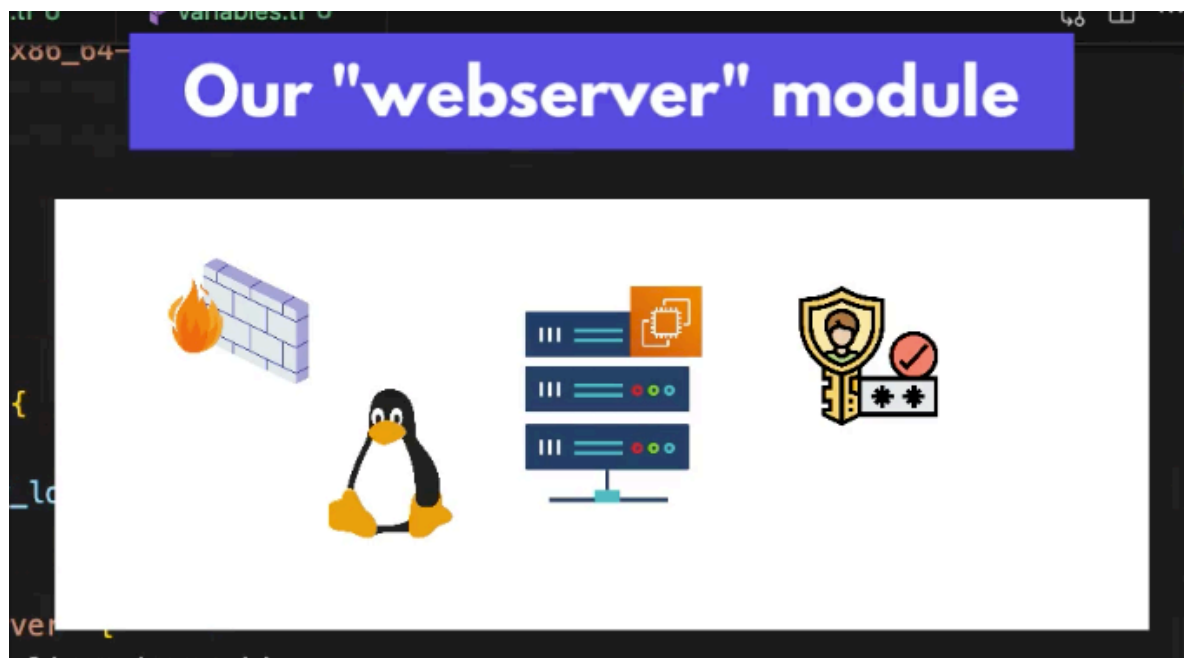
### Modules in Terraform - Part 3

The final modularized project files used in this lecture can be found here:

<https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/modules>

Now let's configure the web server module:

- aws\_instance
- aws\_key\_pair
- aws\_ssm\_parameter -> data (nana uses 'aws\_ami')
- aws\_default\_security\_group



So we need to move 3 resources and one data:

```

1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 resource "aws_vpc" "myapp-vpc" {
6   cidr_block = var.vpc_cidr_block
7   tags = {
8     Name: "${var.env_prefix}-vpc"
9   }
10 }
11
12 module "myapp-subnet" {
13   source = "../modules/subnet"
14   subnet_cidr_block = var.subnet_cidr_block
15   avail_zone = var.avail_zone
16   env_prefix = var.env_prefix
17   vpc_id = aws_vpc.myapp-vpc.id
18   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
19 }
20
21 resource "aws_default_security_group" "default-sg" {
22   vpc_id = var.vpc_id
23 }
24
25 data "aws_ami" "latest-amazon-linux-image" {
26   # most_recent = true
27   # owners = ["amazon"]
28   # filter {
29     #   name = "name"
30     #   values = ["al2023-ami-*kernel-*x86_64"] // it returns a list of values
31     #   values = ["al2023-ami-2023-*kernel-*x86_64"] // it returns a list of values
32   }
33   # filter {
34     #   name = "virtualization-type"
35     #   values = ["hvm"]
36   }
37 }
38
39 data "aws_ssm_parameter" "latest-amazon-linux-image" {
40   name = var.image_name
41 }
42
43 // let EC2 instance know the public key to use for when user logs in
44 resource "aws_key_pair" "ssh-key" {
45   key_name = "ssh-key"
46   public_key = var.ssh_public_key
47 }
48
49 resource "aws_instance" "myapp-server" {
50   ami = data.aws_ami.latest-amazon-linux-image.id
51   instance_type = var.instance_type
52   subnet_id = module.myapp-subnet.subnet_id
53   vpc_security_group_ids = [aws_default_security_group.default-sg.id]
54 }

```

So now I refactor the code to use variables for those external references in web server/main.tf that used to be internal references in root main.tf

See line 2

```

1 resource "aws_default_security_group" "default-sg" {
2   vpc_id = var.vpc_id
3 }
4
5 ingress {
6   from_port = 22
7   to_port = 22
8   protocol = "tcp"
9   cidr_blocks = [var.subnet_cidr_block]
10 }

```

Also we can use a variable instead of a hardcoded image name

```

44
45 data "aws_ssm_parameter" "latest-amazon-linux-image" {
46   name = var.image_name
47   # name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
48 }

```

We move the aws\_instance code to a submodule so now the aws\_instance doesn't have access to the 'module' resource named myapp-subnet in root main.tf so we need to replace it with a variable as well see line 60 📌

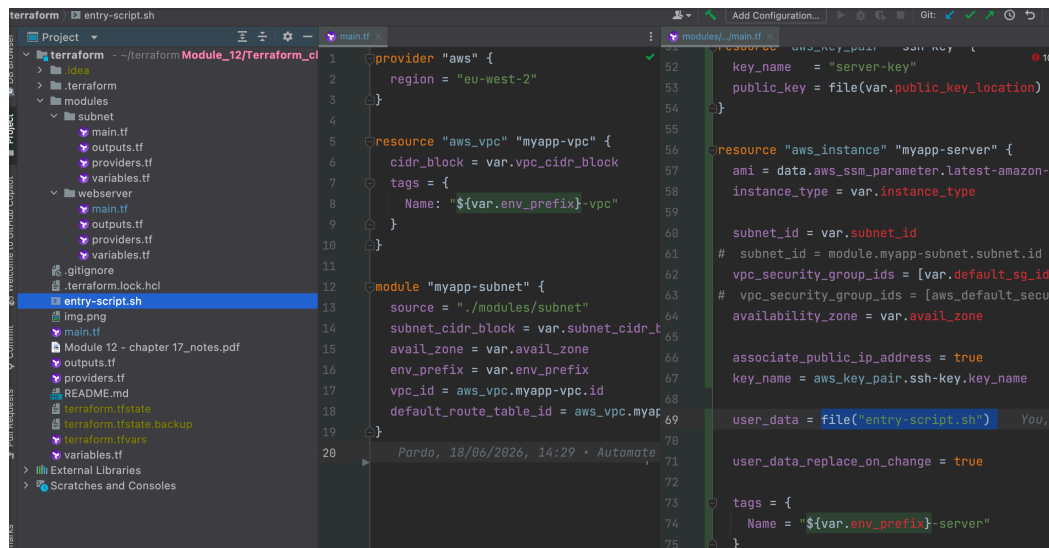
```

56 resource "aws_instance" "myapp-server" {
57   ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
58   instance_type = var.instance_type
59
60   subnet_id = var.subnet_id
61   # subnet_id = module.myapp-subnet.subnet_id
62 }

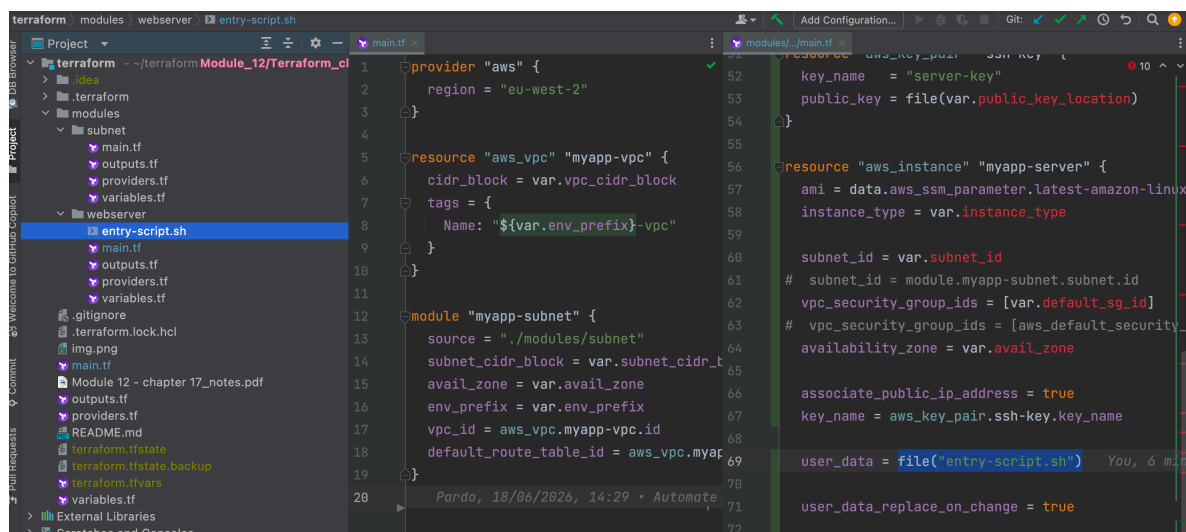
```

And move the script file from the root folder to the webserver submodule, so

line 69 in webserver/main.tf can find it 🙏

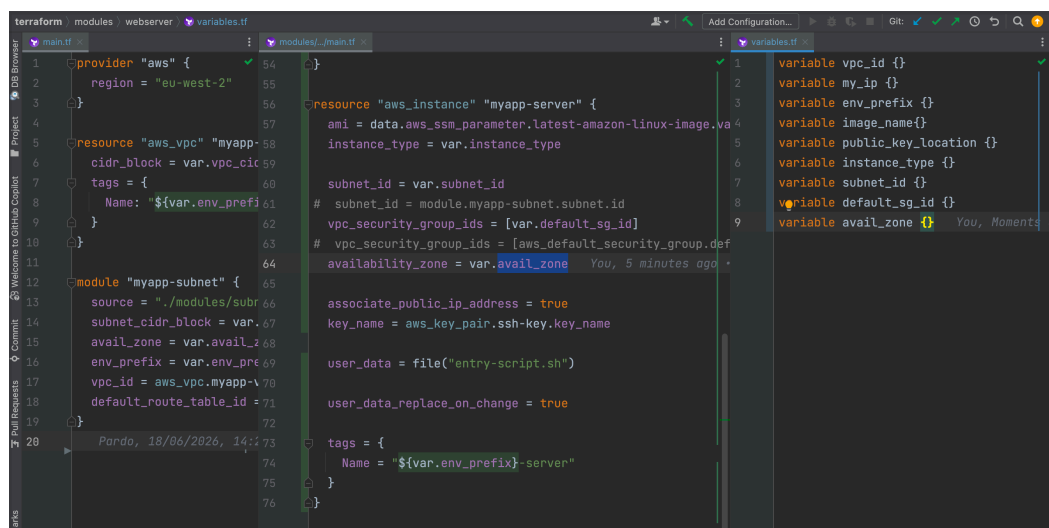


```
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 resource "aws_vpc" "myapp-vpc" {
6   cidr_block = var.vpc_cidr_block
7   tags = {
8     Name = "${var.env_prefix}-vpc"
9   }
10 }
11
12 module "myapp-subnet" {
13   source = "../modules/subnet"
14   subnet_cidr_block = var.subnet_cidr_block
15   avail_zone = var.avail_zone
16   env_prefix = var.env_prefix
17   vpc_id = aws_vpc.myapp-vpc.id
18   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
19 }
20
21 resource "aws_instance" "myapp-server" {
22   key_name = "server-key"
23   public_key = file(var.public_key_location)
24
25   ami = data.aws_ssm_parameter.latest-amazon-linux-image.ami
26   instance_type = var.instance_type
27
28   subnet_id = var.subnet_id
29   # subnet_id = module.myapp-subnet.subnet_id
30   vpc_security_group_ids = [var.default_sg_id]
31   # vpc_security_group_ids = [aws_default_security_group.default_security_group_id]
32   availability_zone = var.avail_zone
33
34   associate_public_ip_address = true
35   key_name = aws_key_pair.ssh-key.key_name
36
37   user_data = file("entry-script.sh")
38   user_data_replace_on_change = true
39
40   tags = {
41     Name = "${var.env_prefix}-server"
42   }
43 }
```



```
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 resource "aws_vpc" "myapp-vpc" {
6   cidr_block = var.vpc_cidr_block
7   tags = {
8     Name = "${var.env_prefix}-vpc"
9   }
10 }
11
12 module "myapp-subnet" {
13   source = "../modules/subnet"
14   subnet_cidr_block = var.subnet_cidr_block
15   avail_zone = var.avail_zone
16   env_prefix = var.env_prefix
17   vpc_id = aws_vpc.myapp-vpc.id
18   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
19 }
20
21 resource "aws_instance" "myapp-server" {
22   key_name = "server-key"
23   public_key = file(var.public_key_location)
24
25   ami = data.aws_ssm_parameter.latest-amazon-linux-image.ami
26   instance_type = var.instance_type
27
28   subnet_id = var.subnet_id
29   # subnet_id = module.myapp-subnet.subnet_id
30   vpc_security_group_ids = [var.default_sg_id]
31   # vpc_security_group_ids = [aws_default_security_group.default_security_group_id]
32   availability_zone = var.avail_zone
33
34   associate_public_ip_address = true
35   key_name = aws_key_pair.ssh-key.key_name
36
37   user_data = file("entry-script.sh")
38   user_data_replace_on_change = true
39
40   tags = {
41     Name = "${var.env_prefix}-server"
42   }
43 }
```

Now let's declare the variables in the webserver/variables.tf file



```
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 resource "aws_vpc" "myapp-vpc" {
6   cidr_block = var.vpc_cidr_block
7   tags = {
8     Name = "${var.env_prefix}-vpc"
9   }
10 }
11
12 module "myapp-subnet" {
13   source = "../modules/subnet"
14   subnet_cidr_block = var.subnet_cidr_block
15   avail_zone = var.avail_zone
16   env_prefix = var.env_prefix
17   vpc_id = aws_vpc.myapp-vpc.id
18   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
19 }
20
21 resource "aws_instance" "myapp-server" {
22   key_name = "server-key"
23   public_key = file(var.public_key_location)
24
25   ami = data.aws_ssm_parameter.latest-amazon-linux-image.ami
26   instance_type = var.instance_type
27
28   subnet_id = var.subnet_id
29   # subnet_id = module.myapp-subnet.subnet_id
30   vpc_security_group_ids = [var.default_sg_id]
31   # vpc_security_group_ids = [aws_default_security_group.default_security_group_id]
32   availability_zone = var.avail_zone
33
34   associate_public_ip_address = true
35   key_name = aws_key_pair.ssh-key.key_name
36
37   user_data = file("entry-script.sh")
38   user_data_replace_on_change = true
39
40   tags = {
41     Name = "${var.env_prefix}-server"
42   }
43 }
```

Now we can reference the module in the root main.tf file

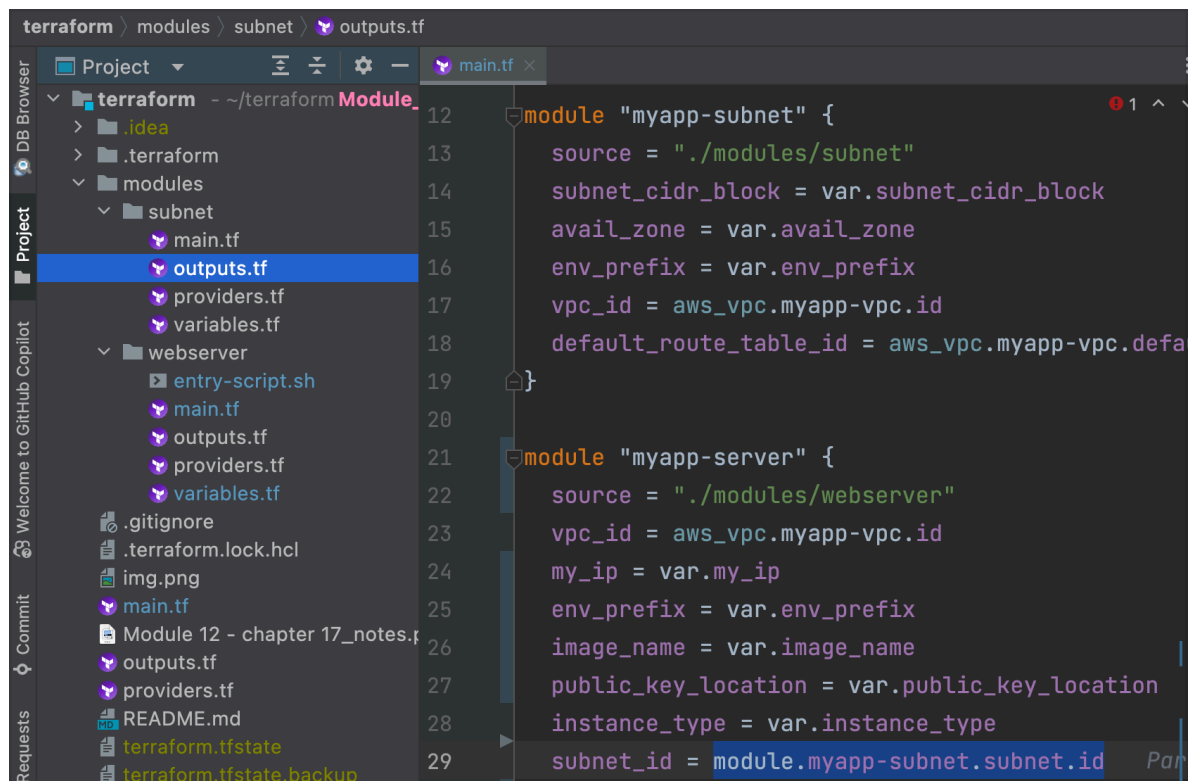
The screenshot shows the Terraform IDE interface. On the left, the 'main.tf' file is open, showing two modules: 'myapp-subnet' and 'myapp-server'. The 'myapp-server' module is currently selected. On the right, the 'variables.tf' file is open, showing a list of variables. The variable 'image\_name' is highlighted in blue. The 'myapp-server' module in 'main.tf' is using 'var.image\_name' for the 'image\_name' attribute of the 'aws\_instance' resource.

I add 'image\_name' to the root variables.tf file (and get rid of the private key var as this was used with provisioners in chapter 15 but we don't need it anymore)

The screenshot shows the Terraform IDE interface after changes. The 'variables.tf' file on the right now includes 'variable image\_name {}' and 'variable private\_key\_location {}'. The 'main.tf' file on the left shows the 'myapp-server' module using 'var.image\_name' for the 'image\_name' attribute of the 'aws\_instance' resource. The 'private\_key\_location' variable is no longer used in the code.

For the subnet\_id attribute we can reference the module "myapp-subnet" because it is created in the same file and then we get the output called 'subnet' to get the object and then we get its id

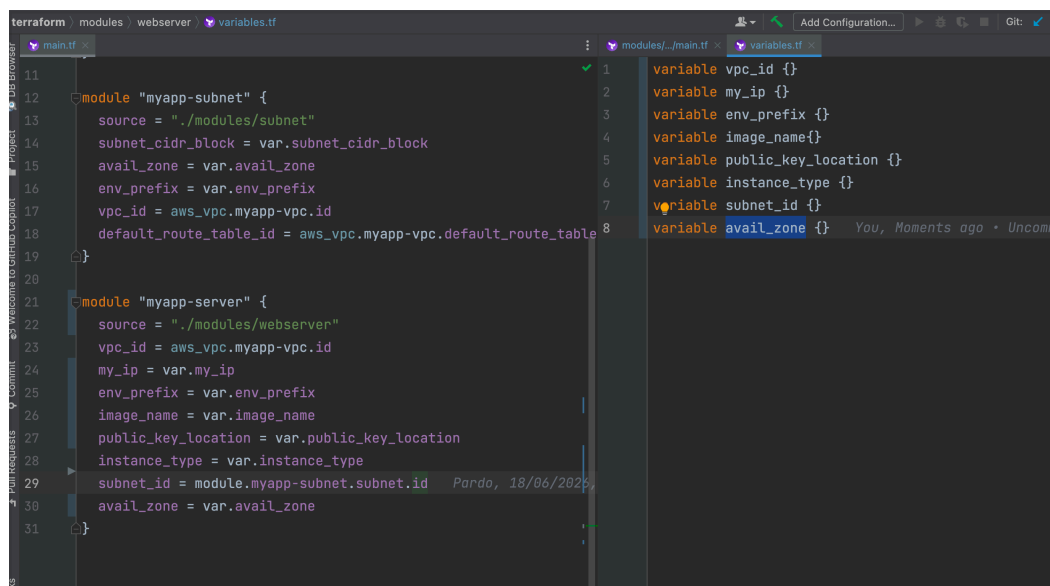
-> module.myapp-subnet.subnet.id



The screenshot shows the VS Code editor with the Terraform module 'myapp-subnet' open in the 'outputs.tf' file. The left sidebar shows the project structure with 'modules/subnet/outputs.tf' selected. The main editor displays the following code:

```
12 module "myapp-subnet" {
13     source = "./modules/subnet"
14     subnet_cidr_block = var.subnet_cidr_block
15     avail_zone = var.avail_zone
16     env_prefix = var.env_prefix
17     vpc_id = aws_vpc.myapp-vpc.id
18     default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
19 }
20
21 module "myapp-server" {
22     source = "./modules/webserver"
23     vpc_id = aws_vpc.myapp-vpc.id
24     my_ip = var.my_ip
25     env_prefix = var.env_prefix
26     image_name = var.image_name
27     public_key_location = var.public_key_location
28     instance_type = var.instance_type
29     subnet_id = module.myapp-subnet.subnet.id
```

And this is the result (see we don't have 'default\_sg\_id' var any longer because we didn't need to replace the reference with a variable in the first place)



The screenshot shows the VS Code editor with the Terraform module 'myapp-server' open in the 'variables.tf' file. The left sidebar shows the project structure with 'modules/webserver/variables.tf' selected. The main editor displays the following code:

```
11 module "myapp-subnet" {
12     source = "./modules/subnet"
13     subnet_cidr_block = var.subnet_cidr_block
14     avail_zone = var.avail_zone
15     env_prefix = var.env_prefix
16     vpc_id = aws_vpc.myapp-vpc.id
17     default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
18 }
19
20 module "myapp-server" {
21     source = "./modules/webserver"
22     vpc_id = aws_vpc.myapp-vpc.id
23     my_ip = var.my_ip
24     env_prefix = var.env_prefix
25     image_name = var.image_name
26     public_key_location = var.public_key_location
27     instance_type = var.instance_type
28     subnet_id = module.myapp-subnet.subnet.id
29     avail_zone = var.avail_zone
30 }
31
```

On the right, the 'variables.tf' file is open, showing the following variables:

```
1 variable vpc_id {}
2 variable my_ip {}
3 variable env_prefix {}
4 variable image_name {}
5 variable public_key_location {}
6 variable instance_type {}
7 variable subnet_id {}
8 variable avail_zone {}
```

Now let's make sure the values of the variables are set in the terraform.tfvars

We need to add image\_name

```

11 module "myapp-subnet" {
12   source = "../modules/subnet"
13   subnet_cidr_block = var.subnet
14   avail_zone = var.avail_zone
15   my_ip = "140.228.47.252/32"
16   env_prefix = var.env_prefix
17   vpc_id = aws_vpc.myapp-vpc.id
18   default_route_table_id = aws_vpc
19 }
20
21 module "myapp-server" {
22   source = "../modules/webserver"
23   vpc_id = aws_vpc.myapp-vpc.id
24   my_ip = var.my_ip
25   env_prefix = var.env_prefix
26   image_name = var.image_name
27   public_key_location = var.publi
28   instance_type = var.instance_ty
29   subnet_id = module.myapp-subnet
30   avail_zone = var.avail_zone
31 }
32
33 vpc_cidr_block = "10.0.0.0/16"
34 subnet_cidr_block = "10.0.10.0/24"
35 avail_zone = "eu-west-2a"
36 env_prefix = "dev"
37 my_ip = "140.228.47.252/32"
38 instance_type = "t2.micro"
39 public_key_location = "/Users/astriid/.ssh/id_ed25519.p
40 image_name = "/aws/service/ami-amazon-linux-latest/al2
41
42 tags = {
43   Name: "${var.env_prefix}-default
44 }
45
46 data "aws_ssm_parameter" "latest-ame
47   name = var.image_name
48   # name = "/aws/service/ami-amazon-l
49 }
50
51 // let EC2 instance know the public
52 resource "aws_key_pair" "ssh-key" {
53   key_name = "server-key"
54   public_key = file(var.public_key_lo
55 }
56
57 resource "aws_instance" "myapp-serve
58   ami = data.aws_ssm_parameter.lates
59   instance_type = var.instance_type
60   subnet_id = var.subnet_id

```

Now we need to review the root outputs.tf file

```

1 output "ec2_public_ip" {
2   value = aws_instance.myapp-server.public_ip
3 }
4
5 #output "latest-amazon-linux-image-id" {
6   # value = data.aws_ami.latest-amazon-linux-image.id
7 }

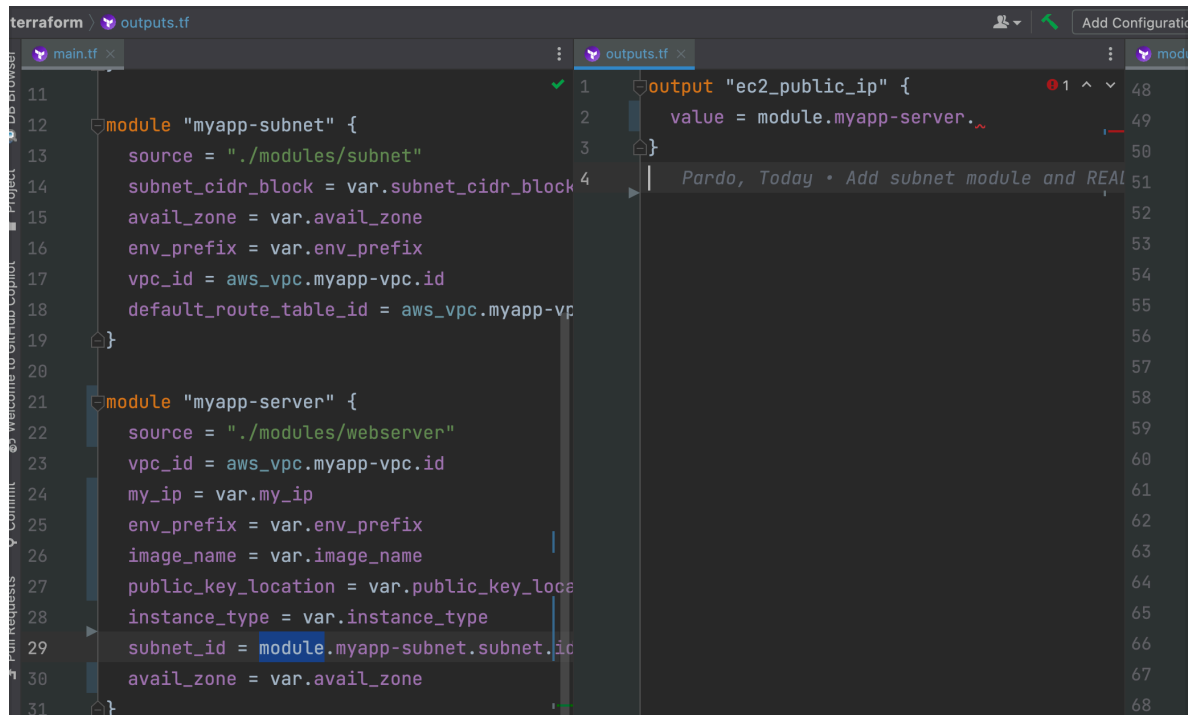
```

I have one output -> Nana uses two outputs but I use 'aws\_ssm\_parameter' so I can't get an output from data.

The outputs don't work any more because the resources that returns them have been moved to submodules.

So I need to update the way we reference the output, with this syntax  
-> module.<module\_name>.<output\_name>.<object\_attribute>

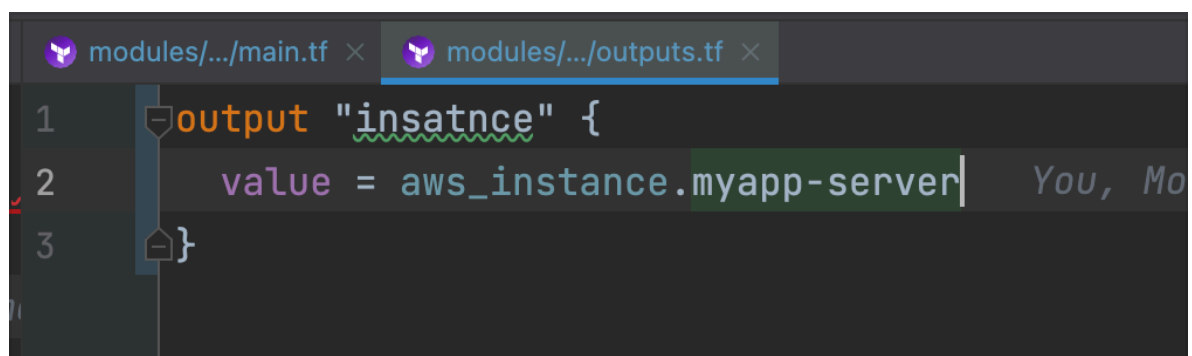
Below we have module.<module\_name> (see line 2 root outputs.tf 🙏)



The screenshot shows a Terraform code editor with two files open: `main.tf` and `outputs.tf`. The `main.tf` file contains two module definitions: `myapp-subnet` and `myapp-server`. The `myapp-server` module uses `aws_instance` to create an EC2 instance. The `outputs.tf` file shows an `output` block for `ec2_public_ip` that references `module.myapp-server`.

```
11 module "myapp-subnet" {
12   source = "../modules/subnet"
13   subnet_cidr_block = var.subnet_cidr_block
14   avail_zone = var.avail_zone
15   env_prefix = var.env_prefix
16   vpc_id = aws_vpc.myapp-vpc.id
17   default_route_table_id = aws_vpc.myapp-vp
18 }
19
20
21 module "myapp-server" {
22   source = "../modules/webserver"
23   vpc_id = aws_vpc.myapp-vpc.id
24   my_ip = var.my_ip
25   env_prefix = var.env_prefix
26   image_name = var.image_name
27   public_key_location = var.public_key_loc
28   instance_type = var.instance_type
29   subnet_id = module.myapp-subnet.subnet.id
30   avail_zone = var.avail_zone
31 }
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
```

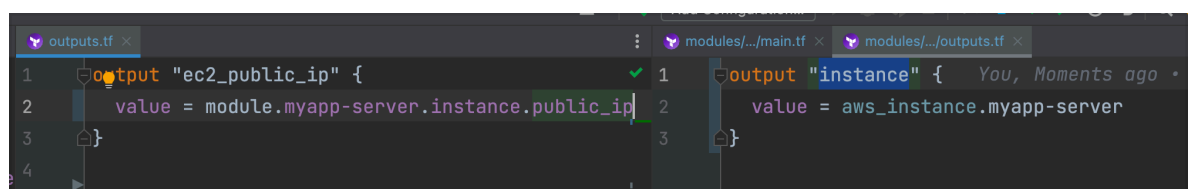
We need to create the output of the werserver/main.tf in the werserver/ outputs.tf and we decided to output the whole instance object



The screenshot shows the `outputs.tf` file for the `webserver` module. It contains an `output` block named `instance` that references `aws_instance.myapp-server`.

```
1 output "instance" {
2   value = aws_instance.myapp-server
3 }
```

And now we can complete the reference  
-> module.myapp-server.instance.public\_ip



The screenshot shows the `outputs.tf` file for the main module. It contains an `output` block named `ec2_public_ip` that references `module.myapp-server.instance.public_ip`.

```
1 output "ec2_public_ip" {
2   value = module.myapp-server.instance.public_ip
3 }
```

Now we execute  
-> terraform init

```

) terraform plan

Error: Module not installed

on main.tf line 21:
21: module "myapp-server" {

This module is not yet installed. Run "terraform init" to install all modules required by this configuration.

) terraform init
Initializing modules...
- myapp-server in modules/webserver
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

Initializing provider plugins found in the state...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,

```

And now we can check the configuration with  
-> terraform plan

```

) terraform plan
~/terraform Module_12/Terraform_chapter-18 +2 !6 terraform plan
Error: Invalid function argument

on modules/webserver/main.tf line 67, in resource "aws_instance" "myapp-server":
67:   user_data = file("entry-script.sh")
    |           |
    |           +-- while calling file(path)

Invalid value for "path" parameter: no file exists at "entry-script.sh"; this function works only with files that are distributed as
part of the configuration source code, so if this file will be created by a resource in this configuration you must instead obtain
this result from an attribute of that resource.

```

Path error 🙅

Then I tried  
-> ./entry-script.sh

```

~/terraform Module_12/Terraform_chapter-18 +2 !6 terraform plan
Error: Invalid function argument

on modules/webserver/main.tf line 67, in resource "aws_instance" "myapp-server":
67:   user_data = file("./entry-script.sh")
    |           |
    |           +-- while calling file(path)

Invalid value for "path" parameter: no file exists at "./entry-script.sh"; this function works only with files that are distributed as
part of the configuration source code, so if this file will be created by a resource in this configuration you must instead obtain
this result from an attribute of that resource.

```

Path error 🙅 again

### Problem:

The ./ in file() resolves relative to where you run terraform plan (the root), not relative to the module file. So it can't find entry-script.sh inside modules/webserver/.

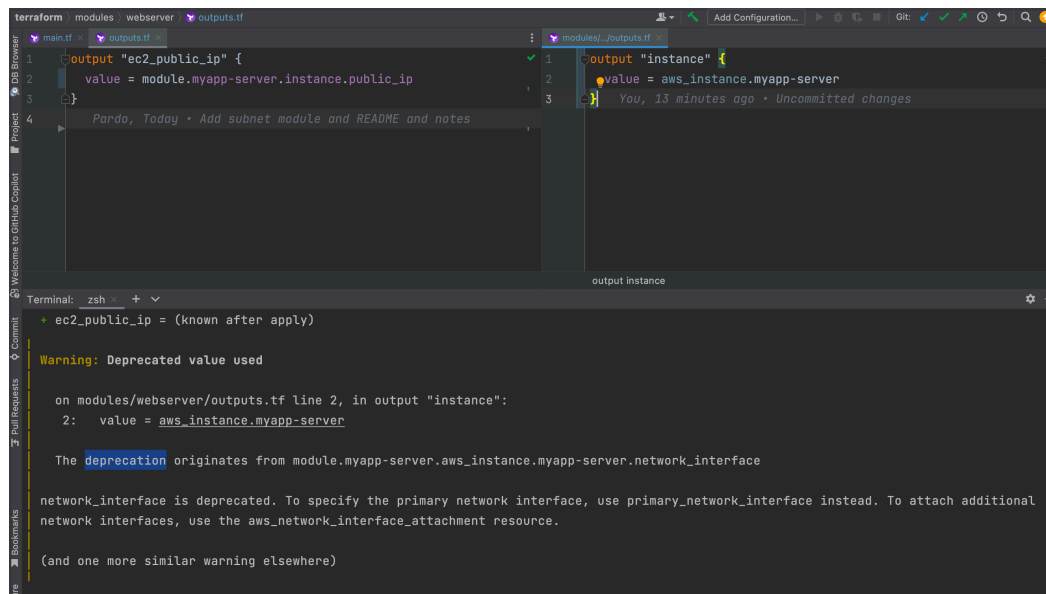


**Fix** — use `path.module`, which always points to the module's own directory:

```
user_data = file("${path.module}/entry-script.sh")
```

Make sure `entry-script.sh` is sitting next to `modules/webserver/main.tf`.

Now I get this warning



## Problem

The warning is because you're outputting the entire `aws_instance` resource, which includes the deprecated `network_interface` attribute.

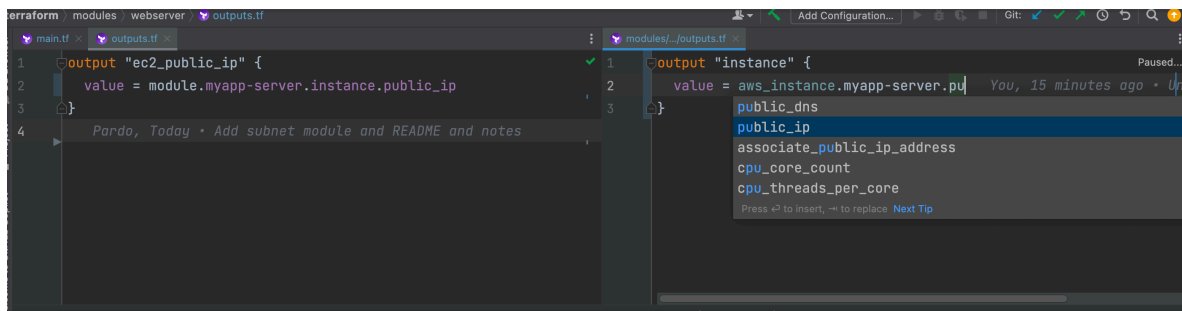
## Fix

Since you only need `public_ip`, output just that attribute instead of the whole instance.

```
modules/webserver/outputs.tf:
output "instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
```

```
Root outputs.tf:
output "ec2_public_ip" {
  value = module.myapp-server.instance_public_ip
}
```

This removes the deprecated attribute from the output entirely and is cleaner — only expose what you actually need from a module.

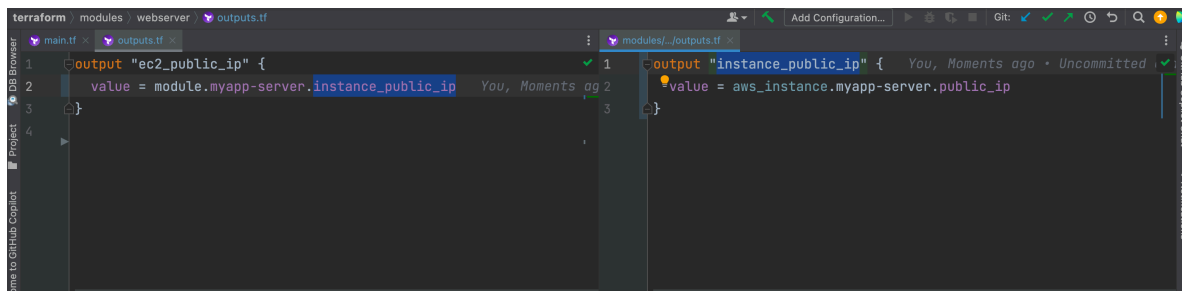


```
1 output "ec2_public_ip" {
2   value = module.myapp-server.instance.public_ip
3 }
4
```

```
1 output "instance" {
2   value = aws_instance.myapp-server.pu
3 }
```

- public\_dns
- public\_ip
- associate\_public\_ip\_address
- cpu\_core\_count
- cpu\_threads\_per\_core

And let's rename the output from 'instance' to 'instance\_public\_ip' as it is more descriptive

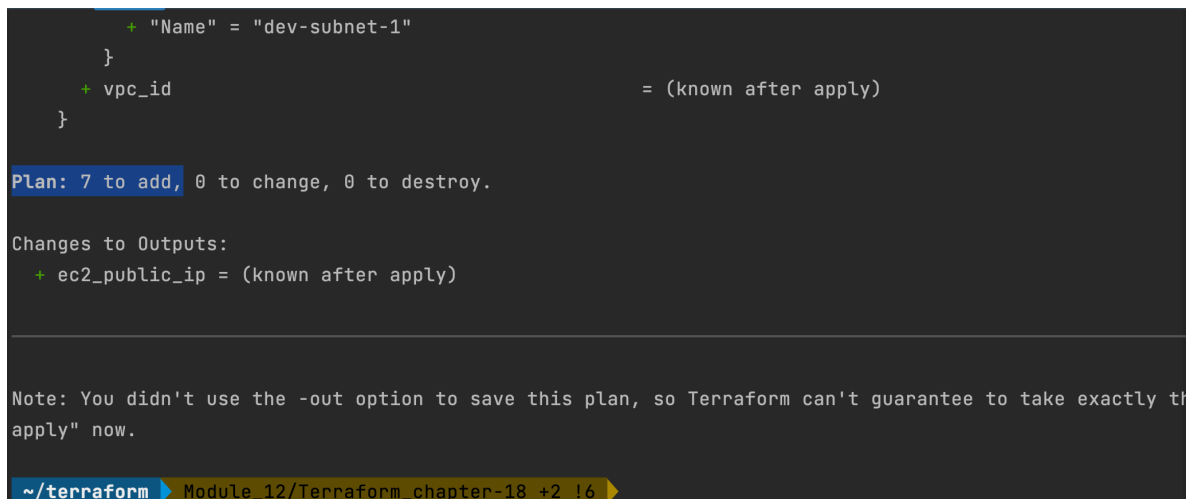


```
1 output "ec2_public_ip" {
2   value = module.myapp-server.instance_public_ip
3 }
4
```

```
1 output "instance_public_ip" {
2   value = aws_instance.myapp-server.public_ip
3 }
4
```

-> terraform plan

And no errors



```
+ "Name" = "dev-subnet-1"
}
+ vpc_id = (known after apply)
}

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly this plan if you "terraform apply" now.

~/terraform Module_12/Terraform_chapter-18 +2 !6
```

-> terraform apply

```

Changes to Outputs:
  + ec2_public_ip = (known after apply)
module.myapp-server.aws_key_pair.ssh-key: Creating...
aws_vpc.myapp-vpc: Creating...
module.myapp-server.aws_key_pair.ssh-key: Creation complete after 0s [id=server-key]
aws_vpc.myapp-vpc: Creation complete after 1s [id=vpc-07fa3069a0c6606d7]
module.myapp-subnet.aws_internet_gateway.myapp-igw: Creating...
module.myapp-subnet.aws_subnet.myapp-subnet-1: Creating...
module.myapp-server.aws_default_security_group.default-sg: Creating...
module.myapp-subnet.aws_internet_gateway.myapp-igw: Creation complete after 1s [id=igw-09ca72f667b49d0c1]
module.myapp-subnet.aws_default_route_table.main-rtb: Creating...
module.myapp-subnet.aws_subnet.myapp-subnet-1: Creation complete after 1s [id=subnet-0a855b8c25c45650c]
module.myapp-subnet.aws_default_route_table.main-rtb: Creation complete after 0s [id=rtb-0ddc3ce1193d09b15]
module.myapp-server.aws_default_security_group.default-sg: Creation complete after 3s [id=sg-0c047838dd28ef732]
module.myapp-server.aws_instance.myapp-server: Creating...
module.myapp-server.aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-server.aws_instance.myapp-server: Still creating... [00m20s elapsed]
module.myapp-server.aws_instance.myapp-server: Creation complete after 22s [id=i-077a7526c16ece317]

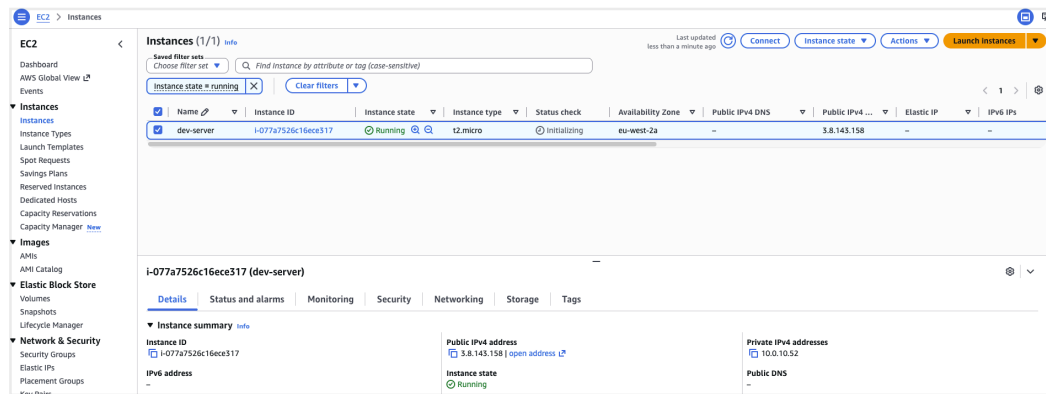
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

ec2_public_ip = "3.8.143.158"

```

And we check the AWS UI



| Name       | Instance ID         | Instance state | Instance type | Status check | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elastic IP | IPv6 IPs |
|------------|---------------------|----------------|---------------|--------------|-------------------|-----------------|-----------------|------------|----------|
| dev-server | i-077a7526c16ece317 | Running        | t2.micro      | initializing | eu-west-2a        | -               | 3.8.143.158     | -          | -        |

| Instance summary       |                     |                     |                            |
|------------------------|---------------------|---------------------|----------------------------|
| Instance ID            | i-077a7526c16ece317 | Public IPv4 address | 3.8.143.158   open address |
| IPv6 address           | -                   | Instance state      | Running                    |
| Private IPv4 addresses | 10.0.10.32          | Public DNS          | -                          |

And it works 🤗

Let's ssh into the EC2 instance and check if the script file run properly and installed docker

```
> ssh ec2-user@3.8.143.158
```

```
The authenticity of host '3.8.143.158 (3.8.143.158)' can't be established.  
ED25519 key fingerprint is SHA256:GD7+6eAoJJwk770XskHJKxKCzh//n0/pQ/UMIUDx/tw.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '3.8.143.158' (ED25519) to the list of known hosts.
```

```

#_
~\_ ##### Amazon Linux 2023
~\_ #####\
~\_ \###|
~\_ \#/ _____
~\_ V~' ' ->
~
~
~_._
~\_/ \_/
~/m/'

```

```
[ec2-user@ip-10-0-10-52 ~]$ docker -v
Docker version 25.0.14, build 0bab007
[ec2-user@ip-10-0-10-52 ~]$
```

## Wrap up

## root module

## "webserver" module

## "subnet" module

## We learnt



## How to write a module



## How to use/reference a module

